Toward Autonomous Web Vulnerability Scanning with Deep Reinforcement Learning

Author 1 Name Surname
Department / School
University / Organization
City, Country
author1@example.com

Author 2 Name Surname
Department / School
University / Organization
City, Country
author2@example.com

Author 3 Name Surname
Department / School
University / Organization
City, Country
author3@example.com

Author 4 Name Surname
Department / School
University / Organization
City, Country
author4@example.com

Author 5 Name Surname
Department / School
University / Organization
City, Country
author5@example.com

Author 6 Name Surname
Department / School
University / Organization
City, Country
author6@example.com

Abstract- Automated web vulnerability scanning remains effective for common signatures but weak on multi-step, context-dependent attack paths. This paper investigates whether deep reinforcement learning can support more adaptive web security testing. We model scanning as a Markov Decision Process and implement an Extended Double Dueling Deep Q-Network (D3QN) agent inside a custom Gymnasium environment, WebSecurityGym. The framework combines Double Q-learning, dueling networks, prioritized replay, noisy exploration, multi-step returns, and a phase-based curriculum that constrains advanced actions until reconnaissance is completed. We evaluate the system on five deliberately vulnerable mock web applications using repeated runs and source-code-verified ground truth. Under the tested configuration, the agent does not yet produce confirmed findings, which highlights current limitations in action design, state representation, and exploit confirmation.

Keywords- deep reinforcement learning, web vulnerability scanning, reinforcement learning, autonomous penetration testing, web application security

I. INTRODUCTION

Modern web applications expose large attack surfaces and increasingly stateful workflows. Signature-based scanners and conventional DAST tools remain useful for common input-validation issues, but they are less effective when detection requires multi-step interaction, role changes, or context carried across requests. Manual penetration testing addresses these cases better, yet it is time consuming and difficult to scale across repeated assessments.

Reinforcement learning (RL) offers a natural formulation for this problem because a scanner repeatedly chooses actions, observes HTTP responses, and adjusts its next step to maximize long-term reward. Instead of evaluating each request in isolation, an RL agent can treat reconnaissance, probing, and exploitation as a sequential decision process.

This paper presents an exploratory RL-based framework for autonomous web vulnerability scanning. The main contributions are a custom Gymnasium environment that converts web interactions into numerical states, an Extended D3QN agent with a phase-based curriculum, and a repeated evaluation protocol against source-code-verified mock applications. The current evaluation does not yet yield confirmed findings, but it provides a reproducible baseline and highlights the technical gaps that must be resolved before such agents can support practical testing.

II. RELATED WORK

Prior work on web security automation spans rule-based scanners, manual-versus-automated comparisons, and more recent RL-based security agents.

A. Traditional Web Application Security Testing

Traditional web application testing combines automated scanners with expert review. DAST tools probe live systems and remain effective for identifying deployment and input-handling flaws [1], while studies comparing manual and automated testing show that human analysts still outperform scripts on contextual business-logic issues [2], [12]. Specialized tools such as WS-Attacker demonstrate that automation can be powerful in narrowly defined domains, but their behavior is still largely driven by predefined checks [3].

This limitation motivates adaptive approaches. Existing toolkits can enumerate pages quickly and leave recognizable traffic patterns [13], yet they may miss multi-step attacks that depend on prior observations, authorization state, or staged payload selection.

B. Reinforcement Learning in Cybersecurity

RL has become a standard approach for sequential decision-making under uncertainty. The DQN result of Mnih et al. [5] showed that deep networks can learn action values directly from observations, and prioritized replay further improves sample efficiency by revisiting informative transitions [6]. In cybersecurity, DRL has also been applied to intrusion detection and other high-dimensional defensive tasks [4].

These results motivate using RL for attack planning, where actions must be selected in sequence and delayed rewards are common.

C. Advanced DRL Architectures for Penetration Testing

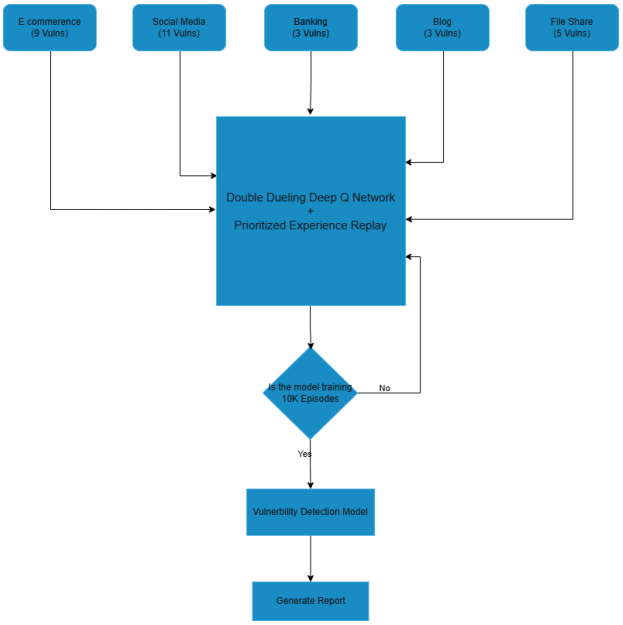
Recent penetration-testing studies have adapted RL to offensive security. Ghanem and Chen modeled network penetration as a sequential decision problem and showed that RL can improve attack-path selection efficiency [7]. Later work explored hierarchical control [8], attack-graph planning [14], continual learning across changing environments [10], hardware-oriented security verification [9], and comparative studies of multiple RL algorithms for automated penetration testing [15].

At the same time, review articles note that current systems still struggle with sparse rewards, partial observability, and generalization across targets [11]. Our work focuses specifically on web applications and uses these findings to motivate a compact state representation, a constrained action curriculum, and an evaluation design centered on confirmed findings rather than raw alerts.

III. METHODOLOGY

The proposed framework separates agent learning from HTTP execution so that the RL policy can be improved without changing the target interaction layer. This section summarizes the system architecture, the MDP formulation, the Extended D3QN design, and the phase-based training strategy.

A. System Architecture



System architecture of the proposed RL-based web vulnerability scanner.

The scanner is organized around three components. WebSecurityGym interfaces with the target application, normalizes observations, and executes payloads selected by the agent. The Extended D3QN policy estimates action values over a discrete payload space. The vulnerable mock applications provide safe training and evaluation targets that expose different vulnerability classes.

The architecture in Fig. 1 supports repeated interaction loops between the target, the environment wrapper, and the RL agent.

1. Target Environment (WebSecurityGym): converts HTTP responses, response timing, and page

structure into a fixed-length state vector and maps discrete actions to concrete requests.

2. Extended D3QN Agent: combines Double Q-learning, dueling heads, prioritized replay, noisy linear layers, and multi-step returns to improve learning stability.
3. Mock Applications: provide isolated training targets with known vulnerabilities and ground-truth labels for evaluation.

B. Formulation of the RL Environment

WebSecurityGym models scanning as an MDP, M = (S, A, P, R, gamma), where each step updates the agent state after a request-response interaction.

1) State Representation (S)

The state vector summarizes features such as HTTP status, response length, latency, form discovery, input reflection, SQL error indicators, and WAF-trigger signals. This compact representation is intended to preserve action-relevant context without encoding raw page content.

- HTTP response metrics: normalized status code, content length, and timing.
- Structural indicators: discovered forms, reflected input, and newly reachable endpoints.
- Security context: database error signatures, authorization failures, and defensive responses.

2) Action Space (A)

The action space contains 150 discrete operations grouped by scanning phase.

- Reconnaissance actions: endpoint discovery, parameter enumeration, and session establishment.
- Assessment actions: low-impact probes that test for anomalous behavior or reflection.
- Exploitation actions: higher-risk payloads targeting vulnerabilities such as SQL injection and cross-site scripting.

C. Agent Design and Q-Learning Mathematics

The policy network is based on Double Dueling DQN. Standard Q-learning updates the action value estimate according to the Bellman equation:

$$Q(S*t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R*t+1+\gamma\max*a' Q(S*t+1, a') - Q(S_t, A_t)]$$

To reduce overestimation, Double DQN separates action selection from target evaluation:

$$Y*t^{\text{DoubleQ}} = R*t+1 + \gamma Q\left(S*t+1, \operatorname{argmax}_a Q(S*t+1, a; \theta_t); \theta_t^-\right)$$

where theta denotes the online network parameters and theta-minus denotes the target network parameters.

A dueling architecture then decomposes state value and action advantage so that the agent can distinguish promising states from the relative quality of candidate actions.

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|}\sum\_a' A(s, a'; \theta, \alpha)\right)$$

Subtracting the mean advantage stabilizes training by constraining the decomposition.

The model is extended with prioritized experience replay, noisy exploration, and multi-step returns. Together, these components improve sample reuse, reduce dependence on manual epsilon scheduling, and accelerate reward propagation from delayed outcomes.

1) Reward Function (R)

The reward function favors verifiable progress while penalizing inefficient or blocked behavior.

- Positive rewards are assigned when the agent triggers a confirmed vulnerability condition or reaches a high-value milestone such as flag capture in controlled scenarios.
- Small step penalties encourage shorter attack chains, while stronger negative rewards discourage rate limits, WAF blocks, and other noisy behavior.

2) Phase-Based Training Algorithm

Training follows a phase-based curriculum. The agent begins with reconnaissance actions only, then unlocks assessment and exploitation actions after sustained reward improvement. This reduces unproductive exploration early in training and encourages the policy to build context before attempting high-impact payloads.

In each episode, the environment is reset, the agent selects an action from the currently unlocked phase, the transition is stored in replay memory, the online network is updated from mini-batches, and the target network is synchronized periodically.

IV. EVALUATION AND RESULTS

The evaluation uses five deliberately vulnerable mock websites and repeated runs under a fixed checkpoint and `mock_targets` configuration. For each target, the scan output is matched against source-code ground truth, and only confirmed findings are counted.

Table I reports per-target averages rather than per-category detail so that the conference version remains compact. Across the stored evaluation artifact, the current agent does not produce confirmed findings on any target under this configuration.

TABLE I. REPEATED EVALUATION RESULTS ACROSS FIVE TARGETS

Target	Existing Cases	Avg. Confirmed Findings	Detection Rate
E-Commerce (5002)	20	0.0	0.0%
Social Media (5003)	20	0.0	0.0%
Banking (5004)	4	0.0	0.0%
Blog (5005)	6	0.0	0.0%
File Share (5006)	6	0.0	0.0%

Although the confirmed-finding rate is zero, this result is still informative. It shows that the present policy can navigate the benchmark but is not yet converting interaction traces into validated exploit chains.

The evaluation therefore functions as a failure analysis as much as a performance test: it identifies sparse rewards, limited payload coverage, and weak contextual reasoning as the main bottlenecks.

V. DISCUSSION

A. Experimental Setup

Experiments were conducted locally with the mock applications hosted on `localhost` to minimize network variance. The agent was trained for 10,000 episodes using mini-batches of 64 and an initial learning rate of 1e-4. Evaluation reuses a fixed trained checkpoint to measure repeatability rather than reporting a single run.

B. Limitations

Several limitations explain the current zero-confirmation result. First, the discrete action space cannot generate or mutate payloads beyond the predefined dictionary. Second, the compact state vector omits some semantic context needed for business logic and authorization flaws. Third, confirmed exploit detection is sparse and delays reward assignment, which makes policy learning difficult. Finally, the current formulation assumes that server responses expose enough state for decision-making, an assumption that weakens on JavaScript-heavy or partially observable applications.

VI. CONCLUSION

This paper presented an RL framework for autonomous web vulnerability scanning based on WebSecurityGym and an Extended D3QN agent. The framework integrates phased action unlocking, replay prioritization, noisy exploration, and a compact state representation for web interactions.

The current evaluation shows that the tested checkpoint does not yet achieve confirmed vulnerability detection on the benchmark targets. Rather than supporting strong performance claims, the results provide a reproducible baseline and identify where the approach is presently limited.

Future work should expand payload generation, improve state modeling with richer semantic context, strengthen exploit confirmation logic, and compare against conventional scanners under the same benchmark. These steps are necessary before RL-based scanners can be considered practical assistants for web application assessment.

ACKNOWLEDGMENT

The authors used AI-assisted language editing during manuscript revision. All technical claims, citations, results, and final wording were reviewed and verified by the authors.

REFERENCES

[1] R. Singh, M. K. Gupta, D. R. Patil, and S. M. Patil, "Analysis of Web Application Vulnerabilities using Dynamic Application Security Testing," in Proc. IEEE 9th Int. Conf. Convergence in Technology (I2CT), 2024, pp. 1-6, doi: 10.1109/I2CT61223.2024.10543484.

[2] R. Sri Devi and M. Mohan Kumar, "Testing for Security Weakness of Web Applications using Ethical Hacking," in Proc. 4th Int. Conf. Trends in Electronics and Informatics (ICOEI), 2020, pp. 354-361, doi: 10.1109/ICOEI48184.2020.9143018.

[3] C. Mainka, J. Somorovsky, and J. Schwenk, "Penetration Testing Tool for Web Services Security," in 2012 IEEE Eighth World Congress on Services, 2012, pp. 163-170, doi: 10.1109/SERVICES.2012.7.

[4] V. Sujatha, K. Lakshmi Prasanna, K. Niharika, V. Charishma, and K. Bhavya Sai, "Network Intrusion Detection using Deep Reinforcement Learning," in Proc. 7th Int. Conf. Computing Methodologies and Communication (ICCMC), 2023, pp. 1146-1150, doi: 10.1109/ICCMC56507.2023.10083673.

[5] V. Mnih et al., "Human-level control through deep reinforcement learning," Nature, vol. 518, no. 7540, pp. 529-533, 2015, doi: 10.1038/nature14236.

[6] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized Experience Replay," arXiv:1511.05952, 2015. [Online]. Available: https://arxiv.org/abs/1511.05952.

[7] M. C. Ghanem and T. M. Chen, "Reinforcement learning for efficient network penetration testing," Information, vol. 11, no. 1, Art. no. 6, 2020, doi: 10.3390/info11010006.

[8] M. C. Ghanem, T. M. Chen, and E. G. Nepomuceno, "Hierarchical reinforcement learning for efficient and effective automated penetration testing of large networks," J. Intell. Inf. Syst., vol. 60, no. 2, pp. 281-303, 2023, doi: 10.1007/s10844-022-00738-0.

[9] H. Al Shaikh, S. Saha, K. Zamiri Azar, F. Farahmandi, M. Tehranipoor, and F. Rahman, "Re-Pen: Reinforcement Learning-Enforced Penetration Testing for SoC Security Verification," IEEE Trans. Very Large Scale Integr. (VLSI) Syst., vol. 33, no. 3, pp. 853-866, 2025, doi: 10.1109/TVLSI.2024.3510682.

[10] S. Zhou, J. Liu, Y. Lu, J. Yang, Y. Zhang, B. Lin, X. Zhong, and S. Hu, "SCRIPT: A Scalable Continual Reinforcement Learning Framework for Autonomous Penetration Testing," Expert Syst. Appl., vol. 285, Art. no. 127827, 2025, doi: 10.1016/j.eswa.2025.127827.

[11] J. Liu, Y. Zhang, S. Zhou, J. Yang, Y. Lu, and X. Zhong, "Autonomous penetration testing using reinforcement learning: A review and perspectives," Expert Syst. Appl., vol. 300, Art. no. 130219, 2026, doi: 10.1016/j.eswa.2025.130219.

[12] N. Singh, V. Meherhomji, and B. R. Chandavarkar, "Automated versus Manual Approach of Web Application Penetration Testing," in Proc. 11th Int. Conf. Computing, Communication and Networking Technologies (ICCCNT), 2020, pp. 1-6, doi: 10.1109/ICCCNT49239.2020.9225385.

[13] D.-Y. Kao, Y.-Y. Chen, and F.-C. Tsai, "Hacking Tool Identification in Penetration Testing," in Proc. 22nd Int. Conf. Advanced Communication Technology (ICACT), 2020, pp. 256-261, doi: 10.23919/ICACT48636.2020.9061401.

[14] A. Chowdhary, D. Huang, J. S. Mahendran, D. Romo, Y. Deng, and A. Sabur, "Autonomous Security Analysis and Penetration Testing," in Proc. 16th Int. Conf. Mobility, Sensing and Networking (MSN), 2020, pp. 508-515, doi: 10.1109/MSN50589.2020.00086.

[15] S. Jaganathan, M. K. Latha, and K. Dharanikota, "Design and analysis of reinforcement learning models for automated penetration testing," IAES Int. J. Artif. Intell., vol. 14, no. 5, pp. 4061-4073, 2025, doi: 10.11591/ijai.v14.i5.pp4061-4073.